

第五章：进程及进程管理

引言

本章导读

本章将不再涉及操作系统框架的大幅调整，而是聚焦于为系统添加进程管理功能。相比之前的内容，本章的学习难度会更为轻松一些。

在完成了页表支持之后，我们对硬件层面的支持已初步完善。然而，目前应用程序的执行方式仍比较固定，缺乏与用户的动态交互。所有测例都是在内核初始化时被一次性加载进内存的，系统运行后无法动态增删应用。从用户的角度看，这与第二章介绍的批处理系统差异不大。

为此，本章我们将实现一个用户终端（Terminal），也称为命令行应用（Command Line Application，常被称为 Shell）。它会构成用户与操作系统交互的命令行界面，其功能与如今常见的命令行工具（如 Linux 中的 bash、Windows 中的 CMD）类似：用户可以输入指令来启动或终止应用，也可以查看系统运行状态。这无疑是现代操作系统中至关重要的一环，它将极大提升系统的可交互性，使用户能更灵活地控制整个系统。

我们可以大致思考一下 Shell 执行命令的过程。首先，它需要能够读取用户输入。如果用户在 Shell 中运行某个测例程序，系统就需要新建一个进程来执行对应的指令流。要注意的是，Shell 本身在系统中也是一个进程。这意味着，我们必须实现进程创建进程的系统调用。事实上，在第四章加入页表支持后，我们已经可以着手实现几个与进程密切相关的系统调用。这些函数在课堂学习中应该已为大家所熟悉：

1. `sys_read(int fd, char* buf, int size)`：从标准输入读取若干字节。
2. `sys_fork()`：创建一个与当前进程几乎完全相同的子进程。
3. `sys_exec(char* filename)`：改变当前进程，使其从头执行指定程序。
4. `sys_wait(int pid, int* exit_code)`：等待指定子进程（或任意子进程）结束，并获取其退出码 `exit_code`。

实践体验

获取本章代码：

```
$ git checkout ch5
```

在 qemu 模拟器上运行本章代码：

```
$ make test BASE=1
```

```
# ....
```

```
app list:
ch2b_exit
ch2b_hello_world
ch2b_power
ch2b_write1
ch3b_sleep
ch3b_sleep1
ch3b_yield0
```

```
ch3b_yield1
ch3b_yield2
ch5b_exec_simple
ch5b_exit
ch5b_forktest0
ch5b_forktest1
ch5b_forktest2
ch5b_getpid
ch5b_usertest
usershell

C user shell
>>
```

不出意外，你将最终运行进入 C user shell，这里，你可以输入 `app list` 中的一个应用，敲击回车之后就可以运行。其中 `ch5b_usertest` 打包了很多应用，只要执行它就能够自动执行所有基础测试：

```
>> ch2b_exit
Shell: Process 2 exited with code 1234
>> ch2b_hello_world
Hello world from user mode program!
Test hello_world OK!
Shell: Process 3 exited with code 0
```

当应用执行完毕后，将继续回到 shell 程序的命令输入模式。另外，这个命令行支持退格键。

本章代码导读

本章中，我们对框架没有做大量代码修改。由于新增的系统调用主要涉及进程相关功能，因此除了在 `syscall.c` 文件中添加了接口定义外，核心函数的具体实现均在 `proc.c` 文件中完成。此外，为了帮助大家更好地理解本章涉及的进程调度相关内容，我们新增了 `queue.c` 文件，其中定义了一个用于管理就绪进程的队列。

在本章练习开始前，我们已经完成了上述几个系统调用的支持。建议各位同学先仔细阅读其实现细节，同时可以结合课堂所学知识进行回顾，这样能大大降低后续练习的难度。

与进程相关的重要系统调用

进程知识回顾

本章实验新增了一系列系统调用，主要对进程结构体、系统调用支持机制以及进程调度相关的数据结构进行了修改。

我们先来回顾一下当前系统中进程支持的状态：

```
UNUSED（未使用）
USED（已使用）
SLEEPING（睡眠）
RUNNABLE（可运行）
RUNNING（运行中）
ZOMBIE（僵尸）
```

其中 `ZOMBIE`（僵尸）状态 在本章实验中可能会首次遇到。在我们的操作系统中，`ZOMBIE` 状态通常出现在以下场景：

当一个进程拥有父进程，且在该父进程尚未结束时自己先结束，此时它会等待父进程回收其资源，在此期间我们将其状态标记为 `ZOMBIE`。

如果对其他部分内容有些遗忘，建议回顾一下第三章的实验内容哦。

说明

进程、线程与协程

进程、线程和协程是操作系统中常见的抽象概念，它们既有联系，也有区别。计算机的核心是 CPU，负责执行各种计算任务；操作系统作为计算机的管理者，则通过进程、线程和协程这些基本单位来管理和调度 CPU，以运行具体的程序。

从历史发展来看，它们出现的顺序依次是：进程 → 线程 → 协程。在还没有进程概念的早期操作系统中，程序在计算机上的一次执行被称为一个“任务”或“作业”，其特点是在整个执行过程中不会被切换。这意味着一个任务必须完全结束后，其他任务才能开始执行，导致系统效率较低。

随着面向 CPU 的时间片切换机制和面向内存的虚拟内存机制的引入，进程的概念应运而生。进程成为 CPU 调度和切换的基本单位，各个进程轮流使用 CPU，并且每个进程拥有独立的内存空间，实现彼此间的地址隔离。此时，操作系统通过进程这一抽象来管理应用程序对 CPU 和内存的使用。

随着对计算机性能要求的提高，进程切换的开销显得较大，于是线程被提出。线程是程序内部的一个顺序执行流程，它是进程的一部分，一个进程可以包含多个线程。线程共享进程的地址空间，但拥有独立的栈（用于函数调用和局部变量）和执行流。线程成为处理器调度和切换的基本单位。线程的管理可以由操作系统内核负责，也可以在用户态的线程库中实现（即“绿色线程”），后者对操作系统而言是不可见的。

协程（也称为纤程）则是更轻量级的执行单元，它建立在线程之上（一个线程内可运行多个协程），并由用户态的协程库进行管理和调度，因此对操作系统透明。多个协程共享同一线程的栈，从而在时间和空间开销上比线程更优。在实现上，协程可以通过用户态库中的函数调用来支持，也可以在语言层面提供原生支持（例如 Rust 的 `async/await` 机制），通过编译器与运行时库的配合来简化编程并提升性能。

核心系统调用详解

fork 系统调用

在操作系统中，每个同时存在的进程都拥有一个独一无二的进程标识符（PID，Process Identifier）。内核完成初始化后，会创建第一个进程，称为初始用户进程（Initial Process）。它是内核中唯一通过硬编码方式直接创建的进程。之后所有的进程，都通过名为 `fork` 的系统调用来产生。

简单来说，创建一个新进程，需要初始化该进程对应的 PCB（进程控制块）结构体、页表、内核栈与用户栈等资源。

```
// os/proc.c
struct proc {
    enum procstate state;      // 进程状态
    int pid;                   // 进程标识符
    pagetable_t pagetable;     // 进程页表
    uint64 ustack;             // 用户栈地址
    uint64 kstack;             // 内核栈地址
    struct trapframe *trapframe; // 中断帧（保存寄存器现场）
    struct context context;     // 进程上下文（用于切换）
    uint64 sz;                  // 进程内存大小
    struct proc *parent;       // 父进程指针
    uint64 exit_code;          // 退出码
};
```

当进程 A 调用 `fork` 系统调用后，内核会创建一个新进程 B，并设定 B 为 A 的子进程。具体实现中，会将 B 的 `parent` 指针指向 A 的 PCB。下面我们通过代码来看 `fork` 是如何完成新进程初始化的：

```
int fork()
{
    struct proc *p = curr_proc();      // 获取当前进程（父进程）
    struct proc *np = allocproc();     // 分配并初始化子进程 PCB
    // 将父进程的用户内存拷贝到子进程
    uvmcopy(p->pagetable, np->pagetable, p->max_page);
    np->max_page = p->max_page;
    // 复制父进程保存的用户寄存器现场（trapframe）
    *(np->trapframe) = *(p->trapframe);
    // 设置子进程中 fork 返回值为 0
    np->trapframe->a0 = 0;
    np->parent = p;                    // 设置父进程指针
    np->state = RUNNABLE;              // 将子进程设为可运行状态
    return np->pid;                    // 父进程返回子进程的 PID
}
```

首先，`fork` 调用 `allocproc` 分配并初始化一个新的进程 PCB（具体过程可回顾前面实验，注意页表的初始化也在 `allocproc` 中完成）。接着，按照 `fork` 的语义，需要将进程 A 的内存内容完整复制到进程 B，使两个进程初始状态一致。

这里不能只简单地复制页表，否则父子进程会指向相同的物理内存页面，导致修改互相干扰，违反进程隔离的原则。正确的做法是：将父进程每一页对应的物理内存都复制一份，并为子进程建立一套映射到这些新物理页的页表。这项工作由 `uvmcopy` 函数完成，它会遍历父进程页表，逐页复制内存内容，并在子进程页表中建立映射。

重要提醒

注意 `mmap` 系统调用对进程 `max_page` 字段的影响。在之前的实验（ch4）中，即使实现有误导致内存泄漏，后果可能还不明显；但在本次实验（lab5）中，此类错误可能会导致严重问题！请务必检查并修正你的 `mmap` 实现。

之后，我们将父进程的 `trapframe`（保存了寄存器状态）复制给子进程，确保子进程能够延续父进程的执行流程。但这里特意将子进程的 `a0` 寄存器（存放返回值）设置为 0，这是为了遵循 `fork` 的约定：在子进程中 `fork` 返回 0。最后，设置子进程的父进程指针和状态。

完成上述步骤后，`fork` 创建的新进程就准备好了。在父进程中，`fork` 的返回值是子进程的 PID。

请大家仔细思考：当我们新创建的子进程 B 第一次被调度执行时，它的执行流程具体是怎样的？这个问题对理解操作系统调度机制非常关键。提示：新进程的 `context` 是如何设置的？`allocproc` 会在进程池中加入一个新进程，那么当它被调度到时，会从哪里开始执行？

wait 系统调用

在 `fork` 建立好父子进程关系之后，`wait` 的实现就比较直观了。我们会通过遍历进程池数组的方式，来查找当前进程的所有子进程。首先来看一下这个系统调用的具体定义：

```
/// pid 表示要等待结束的子进程的进程 ID：若为 0 或 -1，则表示等待任意一个子进程结束；
/// status 是用于保存子进程返回值的地址，如果传入 0，则表示不需要保存。
/// 返回值：如果出错则返回 -1；否则返回结束的子进程的进程 ID。
/// 如果子进程存在但尚未运行结束，该系统调用会阻塞等待。
/// 以下情况会导致错误：pid 非法、指定进程不是当前进程的子进程，或 status 不为 0 但地址不合法。
int waitpid(int pid, int *status);
```

接下来我们看一下 `waitpid` 的具体实现代码：

```
int
wait(int pid, int* code)
{
    struct proc *np;
    int havekids;
    struct proc *p = curr_proc();

    for(;;){
        // 遍历进程表，寻找已结束的子进程
        havekids = 0;
        for(np = pool; np < &pool[NPROC]; np++){
            if(np->state != UNUSED && np->parent == p && (pid <= 0 || np->pid == pid)){
                havekids = 1;
                if(np->state == ZOMBIE){
                    // 找到了一个已结束的子进程
                    np->state = UNUSED;
                    pid = np->pid;
                    *code = np->exit_code;
                    return pid;
                }
            }
        }
        if(!havekids){
            return -1;
        }
        p->state = RUNNABLE;
        sched();
    }
}
```

`wait` 的基本思路是：循环遍历进程数组，检查是否存在与给定 `pid` 匹配的子进程。

1. 如果该子进程已结束（处于 `ZOMBIE` 状态），则清理其状态，并按要求返回进程 ID 和退出码。

2. 如果指定进程不存在，或不是当前进程的子进程，则返回错误。
3. 如果子进程存在但尚未结束，就调用 `sched()` 切换至其他进程，等待子进程运行结束。

exec 系统调用

如果只有 `fork`，那么所有进程都只能执行和初始用户进程相同的代码，这显然是不够的。因此我们还需要引入 `exec` 系统调用来执行不同的可执行文件。

`exec` 所做的工作与之前提到的 `bin_loader` 类似，不同之处在于：`exec` 需要先清理并回收当前进程占用的资源（目前主要是内存），然后再加载新程序。

```
int exec(char *name)
{
    int id = get_id_by_name(name);
    if (id < 0)
        return -1;
    struct proc *p = curr_proc();
    uvmunmap(p->pagetable, 0, p->max_page, 1);
    p->max_page = 0;
    loader(id, p);
    return 0;
}
```

在我们的设计中，`exec` 接收一个要执行的测试文件名。系统会根据文件名找到对应的文件 ID。如果文件存在，则先释放当前进程的内存映射。

注意，`trapframe` 和 `trampoline` 页是进程间可复用的（每个进程都一样），因此不会取消它们的映射。而用户实际的数据页会在取消映射的同时释放对应的物理页面。

接着，程序会调用 `loader` 函数来加载新程序。这个 `loader` 函数相比之前章节做了较大的修改，具体内容我们会在下一节说明。

提示

为什么创建进程要拆成两个系统调用，而不是一个？

你可能会问：为了执行不同的程序，为什么不设计一个同时创建新进程并加载可执行文件的系统调用？

实际上，`fork` + `exec` 的组合是经过实践验证的灵活设计。虽然看起来先 `fork` 再 `exec` 会多一次地址空间的拷贝（之后又在 `exec` 中清除），显得有些浪费，但这种拆分带来了更好的灵活性。

- 额外的拷贝开销可以通过 **写时复制 (Copy on Write)** 等技术大幅降低。
- 拆成两个调用后，可以方便地支持 **重定向 (Redirection)**、管道等高级功能。

这也是类 UNIX 系统的典型做法。与之相比，Windows 使用一个复杂的 `CreateProcess` 函数来同时创建进程并加载程序，该函数参数众多，虽然功能集中，但不如 `fork` + `exec` 组合灵活。

支持了 `fork` 和 `exec` 之后，我们就具备了运行 shell 的基本能力。

进程管理的核心数据结构

本节导读

在本节中，我们将介绍本章实验中用来管理和调度进程的核心数据结构。

进程队列

与之前通过遍历进程池来选择进程的调度方式不同，本章我们实现了一个简单的队列结构，专门用于存放和调度所有处于就绪状态的进程。

队列的定义如下：

```
// os/queue.h

struct queue {
    int data[QUEUE_SIZE];
    int front;
    int tail;
    int empty;
};

void init_queue(struct queue *);
void push_queue(struct queue *, int);
int pop_queue(struct queue *);
```

队列的实现较为简单，其大小为 1024。具体实现代码可参考 `queue.c` 文件。我们将在后续部分说明如何在实际调度中使用该队列。

进程调度

进程调度的核心逻辑体现在 `proc.c` 的 `scheduler` 函数中：

```
// os/proc.c

void scheduler()
{
    struct proc *p;
    for (;;) {
        /* 旧版本调度代码（已注释）
        int has_proc = 0;
        for (p = pool; p < &pool[NPROC]; p++) {
            if (p->state == RUNNABLE) {
                has_proc = 1;
                tracef("swtich to proc %d", p - pool);
                p->state = RUNNING;
                current_proc = p;
                swtch(&idle.context, &p->context);
            }
        }
        if(has_proc == 0) {
            panic("all app are over!\n");
        }
        */
    }
}
```



```

    }
    */

    // 新调度方式：从队列中获取任务
    p = fetch_task();
    if (p == NULL) {
        panic("all app are over!\n");
    }
    tracef("swtich to proc %d", p - pool);
    p->state = RUNNING;
    current_proc = p;
    swtch(&idle.context, &p->context);
}
}

```

可以看到，我们不再使用原来遍历整个进程池、并从中挑选就绪进程的简单调度方式，而是改为直接调用 `fetch_task` 函数从就绪队列中取出下一个应当执行的进程，然后进行上下文切换。

对于已经运行结束或时间片用完的进程，我们将其重新加入队列末尾。这种基于队列的调度方式相比原先的方法更加高效，每次调度只需常数时间即可完成。

另外，由于我们使用的是先进先出（FIFO）队列，因此当前框架默认采用的是 FIFO 调度算法。

Shell与测例加载

本节导读

在本节中，我们将介绍如何使用新的 `bin_loader` 将测例加载到进程里，并了解我们的Shell测例是如何运行的。

新的bin_loader

`exec` 函数会调用 `bin_loader`，将指定名称的测例加载到目标进程 `p` 中。请结合以下代码注释来理解 `bin_loader` 的变化：

```

int bin_loader(uint64 start, uint64 end, struct proc *p)
{
    void *page;
    // 注意：现在不要求起始地址对齐。核心逻辑依然是将物理地址范围 [start, end)
    // 映射到虚拟内存的 [BASE_ADDRESS, BASE_ADDRESS + length)
    uint64 pa_start = PGROUNDDOWN(start);
    uint64 pa_end = PGROUNDUP(end);
    uint64 length = pa_end - pa_start;
    uint64 va_start = BASE_ADDRESS;
    uint64 va_end = BASE_ADDRESS + length;

    // 不再一次性映射多个页面，而是逐页映射。为什么要这样做？
    for (uint64 va = va_start, pa = pa_start; pa < pa_end;
        va += PGSIZE, pa += PGSIZE) {
        // 我们不会直接映射物理页，而是先分配一个新页面，再用 memmove 拷贝内容。
        // 这样做可以避免对齐问题，但背后其实有更重要的原因。
        page = kalloc();
        memmove(page, (const void *)pa, PGSIZE);
        // 下面的 if 是为了避免因 start 和 end 未对齐而拷贝多余的内核数据。
    }
}

```



```

// 需要手动将这些部分清零。
if (pa < start) {
    memset(page, 0, start - va);
} else if (pa + PAGE_SIZE > end) {
    memset(page + (end - pa), 0, PAGE_SIZE - (end - pa));
}
mappages(p->pagetable, va, PGSIZE, (uint64)page, PTE_U | PTE_R | PTE_W | PTE_X);
}

// 和 lab4 一样，映射用户栈
p->ustack = va_end + PAGE_SIZE;
for (uint64 va = p->ustack; va < p->ustack + USTACK_SIZE;
     va += PGSIZE) {
    page = kalloc();
    memset(page, 0, PGSIZE);
    mappages(p->pagetable, va, PGSIZE, (uint64)page, PTE_U | PTE_R | PTE_W);
}

// 设置 trapframe
p->trapframe->sp = p->ustack + USTACK_SIZE;
p->trapframe->epc = va_start;
p->max_page = PGROUNDUP(p->ustack + USTACK_SIZE - 1) / PAGE_SIZE;
p->state = RUNNABLE;
return 0;
}

```

其中，用户栈、trapframe 和 trampoline 的映射方式没有变化，但 `.bin` 数据的映射看起来和之前完全不同——它现在是通过一个循环完成的。

其实，这个循环的逻辑很简单：针对 `.bin` 的每一页，分配一个新页并复制内容，然后建立该页的映射。之所以要这样做，主要是因为我们的物理内存管理较为简单，一次只能分配一个页面。如果能分配连续的物理页，这个循环可以用一个 `mappages` 调用来替代。

那么，更关键的问题是：为什么要拷贝内容呢？回顾 lab4，我们当时是直接将虚拟内存映射到物理内存，并没有拷贝。那么现在为什么要多这一步呢？

实际上，如果按照 lab4 的方式，程序运行时会直接修改唯一的“程序原始镜像”。这样会导致程序只能运行一次：第二次执行时，`.data` 和 `.bss` 段的数据已经被上一次执行修改，不再是初始状态。在 lab4 中，每个程序最多执行一次，所以那样做是可行的。但在 lab5 中，所有程序都可能被反复执行，因此我们必须保护“程序原始镜像”，确保每次运行都是基于原始镜像的一份新拷贝。

测例的执行

从本章开始，大家可以发现我们的 `run_all_app` 函数被 `load_init_app` 取代了：

```

// os/loader.c

// load all apps and init the corresponding `proc` structure.
int load_init_app()
{
    int id = get_id_by_name(INIT_PROC);
    if (id < 0)
        panic("Cannpt find INIT_PROC %s", INIT_PROC);
}

```

```

struct proc *p = allocproc();
if (p == NULL) {
    panic("allocproc\n");
}
debugf("load init proc %s", INIT_PROC);
loader(id, p);
return 0;
}

```

这个 `load_init_app` load 的 `INIT_PROC` 一般来说就是我们在本章第一节展示的那个 `usershell`，不过可以通过在 `Makefile` 中传入 `INIT_PROC` 参数而改变，大部分情况下，不推荐修改，这是由于 `usershell` 具有不错的灵活性。

usershell

`user/src/usershell.c` 就是 `usershell` 的代码了，有兴趣的同学可以研究下这个 shell:

```

const unsigned char LF = 0x0a;
const unsigned char CR = 0x0d;
const unsigned char DL = 0x7f;
const unsigned char BS = 0x08;

// 手搓了一个极简的 stack，用来维护用户输入，保存一行的输入
char line[100] = {};
int top = 0;
void push(char c){ line[top++] = c; }
void pop() { --top; }
int is_empty() { return top == 0; }
void clear() { top = 0; }

int main()
{
    printf("C user shell\n");
    printf(">> ");
    fflush(stdout);
    while (1) {
        char c = getchar();
        switch (c) {
            // 回车，执行当前 stack 中字符串对应的程序
            case LF:
            case CR:
                printf("\n");
                if (!is_empty()) {
                    push('\0');
                    int pid = fork();
                    if (pid == 0) {
                        // child process
                        if (exec(line, NULL) < 0) {
                            printf("no such program: %s\n",
                                line);
                            exit(0);
                        }
                    }
                }
            }
        }
    }
}

```

```

        panic("unreachable!");
    } else {
        int xstate = 0;
        int exit_pid = 0;
        exit_pid = waitpid(pid, &xstate);
        assert(pid == exit_pid);
        printf("shell: Process %d exited with code %d\n",
               pid, xstate);
    }
    clear();
}
printf(">> ");
fflush(stdout);
break;
// 退格键, pop一个char
case BS:
case DL:
    if (!is_empty()) {
        putchar(BS);
        printf(" ");
        putchar(BS);
        fflush(stdout);
        pop();
    }
    break;
// 普通输入, 回显并 push 一个 char
default:
    putchar(c);
    fflush(stdout);
    push(c);
    break;
}
}
return 0;
}

```

可以看到这个测例实际上就是实现了一个简单的字符串处理的函数，并且针对解析得到的不同的指令调用不同的系统调用。要注意这需要 shell 支持 read 的系统调用。当读入用户的输入时，它会死循环的等待用户输入一个代表程序名称的字符串(通过 `sys_read`)，当用户按下空格之后，shell 会使用 `fork` 和 `exec` 创建并执行这个程序，然后通过 `sys_wait` 来等待程序执行结束，并输出 `exit_code`。有了 shell 之后，我们可以只执行自己希望的程序，也可以执行某一个程序很多次来观察输出，这对于使用体验是极大的提升！可以说，第五章的所有努力都是为了支持 shell。

我们简单看一下 `sys_read` 的实现，它与 `sys_write` 有点相似：

```

uint64 sys_read(int fd, uint64 va, uint64 len)
{
    if (fd != STDIN)
        return -1;
    struct proc *p = curr_proc();
    char str[MAX_STR_LEN];
    len = MIN(len, MAX_STR_LEN);
    for (int i = 0; i < len; ++i) {

```

```
        // consgetc() 会阻塞式的等待读取一个 char
        int c = consgetc();
        str[i] = c;
    }
    copyout(p->pagetable, va, str, len);
    return len;
}
```

目前我们只支持标准输入 stdin 的输入（对应 fd = STDIN）。

编程作业：进程创建与 Stride 调度

在理解了进程管理和调度机制后，本节将通过两个编程任务来加深对进程创建和调度算法的理解。

作业目标

实现两个核心功能：

功能	系统调用	调用号	说明
进程创建	<code>sys_spawn</code>	400	创建新进程并执行指定程序（类似 fork+exec 但更高效）
优先级设置	<code>sys_set_priority</code>	140	设置进程优先级，配合 Stride 调度算法

任务一：实现 sys_spawn

理解 spawn vs fork+exec

传统 Unix 创建新进程的方式是 fork+exec：

```
fork: 父进程 → 子进程（完整复制内存）
exec: 子进程 → 执行新程序（丢弃刚复制的内存）
```

问题：fork 复制了整个地址空间，但 exec 立即丢弃它，这是浪费！

spawn 的优势：直接创建新进程并加载程序，无需复制父进程内存。

sys_spawn 接口定义

```
/// 功能：创建一个新进程并执行指定程序
/// 参数：path - 要执行的程序路径（用户空间字符串）
/// 返回值：成功返回子进程PID，失败返回 -1
int sys_spawn(uint64 path_va);
```

实现思路

spawn = allocproc + 设置父进程 + init_stdio + bin_loader + add_task

```
// os/syscall.c
```

```

/* ch5: sys_spawn - 创建新进程并执行程序 */
int sys_spawn(uint64 path_va)
{
    struct proc *p = curr_proc();
    char path[MAX_STR_LEN];
    struct inode *ip;
    struct proc *np;

    /* ch5: 从用户空间拷贝路径名 */
    if (copyinstr(p->pagetable, path, path_va, MAX_STR_LEN) < 0)
        return -1;

    /* ch5: 查找可执行文件 */
    if ((ip = namei(path)) == NULL)
        return -1;

    /* ch5: 分配新进程 */
    if ((np = allocproc()) == NULL) {
        iput(ip);
        return -1;
    }

    /* ch5: 设置父进程 */
    np->parent = p;

    /* ch5: 初始化标准IO */
    init_stdio(np);

    /* ch5: 复制父进程的文件描述符表(跳过stdio) */
    for (int i = 3; i < FD_BUFFER_SIZE; i++) {
        if (p->files[i] != NULL) {
            p->files[i]->ref++;
            np->files[i] = p->files[i];
        }
    }

    /* ch5: 加载可执行文件 */
    bin_loader(ip, np);
    iput(ip);

    /* ch5: 设置命令行参数 */
    char *argv[2] = {path, NULL};
    struct thread *nt = &np->threads[0];
    nt->trapframe->a0 = push_argv(np, argv);

    /* ch5: 将新进程加入调度队列 */
    nt->state = RUNNABLE;
    add_task(nt);

    return np->pid;
}

```

任务二：实现 Stride 调度算法

Stride 算法原理

简单的时间片轮转调度对所有进程一视同仁，无法实现"优先级高的进程获得更多CPU时间"。

Stride 算法的核心思想：

每个进程有两个属性：

- `stride`：累计的"虚拟运行时间"
- `priority`：优先级（2-255，越大优先级越高）

调度规则：

1. 每次选择 `stride` 最小的进程运行
2. 运行后更新：`stride += BIG_STRIDE / priority`

数学直觉：优先级高的进程，每次 `stride` 增加得少，更容易被选中。

举例：

进程A: `priority=2`，每次 `stride += 65536/2 = 32768`

进程B: `priority=4`，每次 `stride += 65536/4 = 16384`

初始: `A.stride=0`, `B.stride=0`

第1次: 选A, `A.stride=32768`

第2次: `B.stride=0 < A.stride`, 选B, `B.stride=16384`

第3次: `B.stride=16384 < A.stride`, 选B, `B.stride=32768`

第4次: 两者相等...

结果: A运行1次, B运行2次, 符合 $2:4 = 1:2$ 的比例

扩展进程控制块

```
// os/proc.h
struct proc {
    // ... 原有字段 ...
    /* ch5: stride调度算法所需字段 */
    uint64 stride;    /* 当前已运行的"长度" */
    uint64 priority; /* 进程优先级, 默认16 */
};
```

实现 sys_set_priority

```
// os/syscall.c

/* ch5: sys_set_priority - 设置进程优先级 */
int sys_set_priority(long long prio)
{
    /* ch5: 优先级必须 >= 2 */
    if (prio < 2)
        return -1;

    struct proc *p = curr_proc();
    p->priority = prio;
    return prio;
}
```

修改调度器

```
// os/proc.c

#define BIG_STRIDE 65536

/* ch5: 从任务队列中获取stride最小的任务 */
struct thread *fetch_task()
{
    if (task_queue.empty)
        return NULL;

    /* ch5: 遍历队列找stride最小的任务 */
    int min_idx = -1;
    uint64 min_stride = (uint64)-1;

    // 计算队列中的元素数量
    int count = /* 根据front和tail计算 */;

    for (int i = 0; i < count; i++) {
        int idx = (task_queue.front + i) % task_queue.size;
        struct thread *t = id_to_task(task_queue.data[idx]);
        if (t == NULL || t->state != RUNNABLE)
            continue;

        uint64 stride = t->process->stride;
        if (stride < min_stride) {
            min_stride = stride;
            min_idx = idx;
        }
    }

    /* ch5: 从队列中移除选中的任务并更新stride */
    // ... 移除逻辑 ...

    struct thread *t = id_to_task(task_id);
```



```
    if (t != NULL) {
        t->process->stride += BIG_STRIDE / t->process->priority;
    }

    return t;
}
```

知识点总结

通过本作业，我们深入理解了以下概念：

概念	说明
spawn vs fork+exec	spawn 更高效，因为不需要复制父进程的地址空间
Stride 调度	基于虚拟运行时间的公平调度，保证高优先级进程获得更多 CPU
BIG_STRIDE	常量 65536，用于计算 pass 值，避免整数除法精度问题
优先级范围	≥ 2 ，避免除零错误

测试验证

- **spawn 测试**：运行 `ch5b_usertest`，验证进程创建和执行
- **stride 测试**：运行 `ch5t_usertest`，验证优先级调度比例

修改文件清单

文件	修改内容
<code>os/proc.h</code>	添加 <code>stride</code> 和 <code>priority</code> 字段
<code>os/proc.c</code>	初始化字段，实现 stride 调度的 <code>fetch_task()</code>
<code>os/syscall.c</code>	实现 <code>sys_spawn()</code> 和 <code>sys_set_priority()</code>